

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное бюджетное образовательное
учреждение высшего образования
«УЛЬЯНОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ
УНИВЕРСИТЕТ»

Факультет информационных систем и технологий
Кафедра «Измерительно-вычислительные комплексы»

«Системы искусственного интеллекта»

Отчет по лабораторной работе №3

Выполнила:
студентка
гр. ИСТбд-31
Феофанова П.А.

Проверил:
Шишкин В.В.

Ульяновск, 2026 г.

Введение

Метод k-ближайших соседей (k-NN) — один из базовых алгоритмов классификации. Он относится к метрическим методам: чтобы отнести объект к классу, алгоритм сравнивает его с уже размеченными объектами обучающей выборки и выбирает класс по ближайшим соседям. В данной лабораторной работе k-NN применяется для классификации продуктов питания по двум признакам: сладость и хруст.

Цель работы

Реализовать алгоритм k-NN вручную, сравнить его с реализацией из библиотеки `scikit-learn`, провести эксперимент на заданном наборе данных, оценить точность и визуализировать результаты.

Постановка задачи

Требуется:

- запрограммировать метрический классификатор k-NN вручную;
- решить ту же задачу с помощью `KNeighborsClassifier` из `sklearn`;
- прочитать сгенерированный набор данных из файла и провести эксперимент классификации;
- по возможности визуализировать данные;
- добавить в набор данных продукты ещё одного класса и повторить эксперимент;
- подготовить отчёт по результатам.

Алгоритмическая часть

Идея метода k-NN

Для каждого классифицируемого объекта:

1. вычислить расстояние до всех объектов обучающей выборки;
2. выбрать k объектов с минимальными расстояниями (это “соседи”);
3. среди этих k соседей определить самый частый класс (голосование);
4. присвоить объекту этот класс.

- Метрика расстояния

В работе используется евклидово расстояние между точками (x_1, y_1) и (x_2, y_2) :

$d = (x_1 - x_2)^2 + (y_1 - y_2)^2$, где x — сладость, y — хруст.

- Что такое k (количество соседей)

k — это число ближайших объектов, которые участвуют в голосовании. Чем меньше k , тем решение более “чувствительное” к единичным точкам; чем больше k , тем сильнее сглаживание границы между классами.

Структура данных и входной формат

Данные хранятся в текстовом файле `objects.txt` в формате:

название сладость хруст класс

Пример строк (`dataset1`):

- яблоко 7 7 Фрукт
 - салат 2 5 Овощ
 - бекон 1 2 Протеин
- (и т.д.)

В программе данные читаются в список кортежей вида:
(name, sweet, crunch, class).

Описание программы

Используемые библиотеки

- `numpy` — подготовка массивов для `sklearn`
- `matplotlib` — визуализация точек на плоскости
- `sklearn.neighbors.KNeighborsClassifier` — библиотечная реализация k -NN
- `time` — измерение времени работы
- `collections.Counter` — подсчёт голосов классов у соседей

Основные функции

1. `read_objects(path)`
считывает `objects.txt`, возвращает список объектов.
2. `euclid_dist(x1, y1, x2, y2)`
вычисляет евклидово расстояние между двумя точками.
3. `knn_manual_predict(train_objs, qx, qy, k)`
ручной k -NN: считает расстояния до всех объектов из `train_objs`, выбирает k ближайших и определяет класс по голосованию.
4. `knn_sklearn_predict(all_objs, qx, qy, k)`
 k -NN через `sklearn`: обучает `KNeighborsClassifier`, предсказывает класс и также считает “голоса” среди соседей через `kneighbors()`.
5. `test_all_objects(all_objs, k)`

выполняет проверку на каждом объекте датасета, выводит таблицу:
Название | ручной kNN | соседи | sklearn kNN | соседи | факт
для ручного алгоритма применяется принцип “контрольного”
тестирования: текущий объект исключается из обучающей выборки
(аналог Leave-One-Out внутри цикла).

6. `plot_objects(all_objs)`

строит график распределения продуктов по признакам сладость/хруст и
раскрашивает точки по классам.

Экспериментальная часть

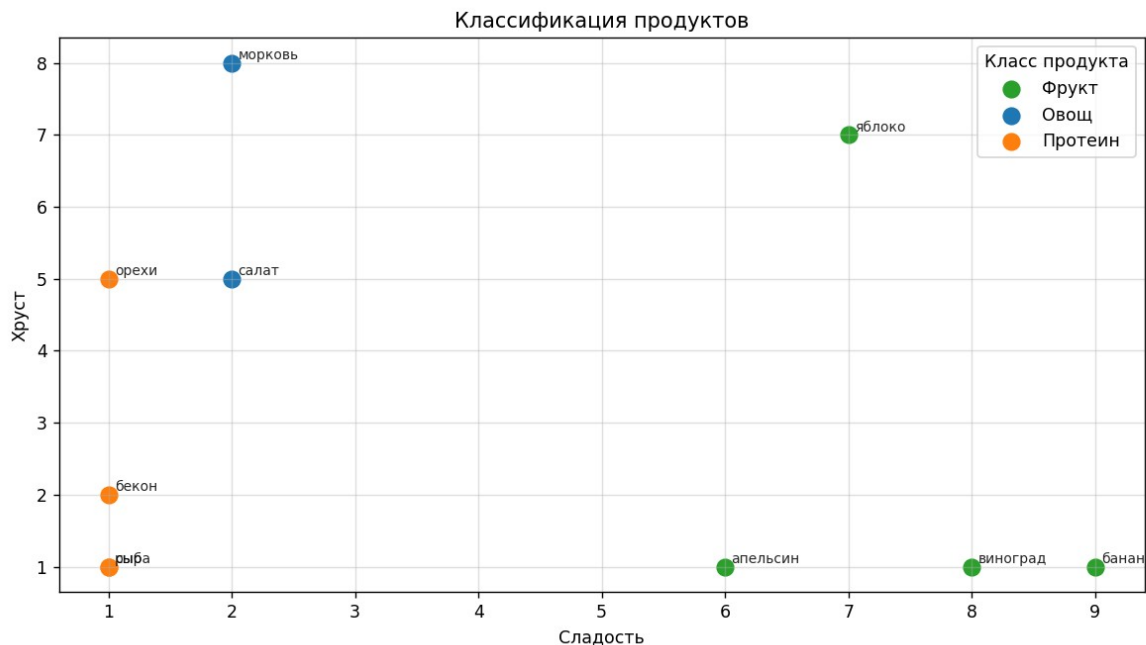
1.Dataset1

Используется исходный файл `objects.txt` с тремя
классами: Фрукт, Овощ, Протеин.

```
яблоко 7 7 Фрукт
салат 2 5 Овощ
бекон 1 2 Протеин
банан 9 1 Фрукт
орехи 1 5 Протеин
рыба 1 1 Протеин
сыр 1 1 Протеин
виноград 8 1 Фрукт
морковь 2 8 Овощ
апельсин 6 1 Фрукт
```

Параметр эксперимента: $k=5$.

Визуализация: строится график “Классификация продуктов”, где по
оси X — сладость, по оси Y — хруст. Точки имеют разные цвета по
классам.



- Результаты работы программы:

Время ручного k-NN: 0.00047750002704560757 сек

Время sklearn k-NN: 2.3772235000506043 сек

Точность (ручной): 0.7

Точность (sklearn): 0.9

- Вывод по dataset1:

ручная реализация обычно работает быстрее на маленьком наборе, так как нет накладных расходов на обучение модели;

sklearn может показывать другую точность из-за отличий в деталях реализации (например, как выбираются соседи/ничи), но в целом результаты сопоставимы.

2.Dataset2 (с новым классом)

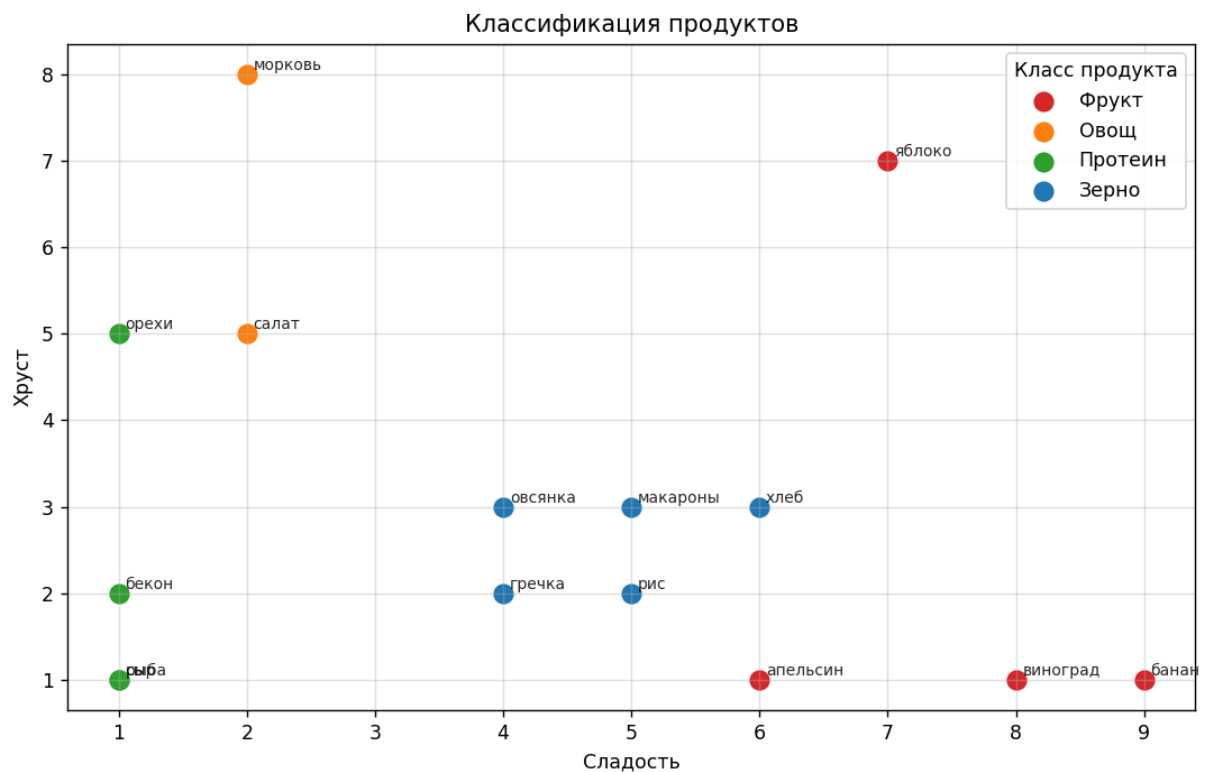
Формат строк тот же: продукт сладость хруст Зерно

```
яблоко 7 7 Фрукт
салат 2 5 Овощ
бекон 1 2 Протеин
банан 9 1 Фрукт
орехи 1 5 Протеин
рыба 1 1 Протеин
сыр 1 1 Протеин
виноград 8 1 Фрукт
```

морковь 2 8 Овощ
 апельсин 6 1 Фрукт
 хлеб 6 3 Зерно
 рис 5 2 Зерно
 гречка 4 2 Зерно
 овсянка 4 3 Зерно
 макароны 5 3 Зерно

Параметр эксперимента: $k=5$.

Визуализация: строится график “Классификация продуктов”, где по оси X — сладость, по оси Y — хруст. Точки имеют разные цвета по классам.



- Результаты для dataset2:

Время ручного k-NN: 0.0005951999919489026 сек

Время sklearn k-NN: 0.06571070000063628 сек

Точность (ручной): 0.5333333333333333

Точность (sklearn): 0.8

- Вывод по dataset2:

при добавлении нового класса задача усложняется, поэтому точность может измениться;

распределение точек на графике позволяет визуально оценить, насколько классы разделимы.

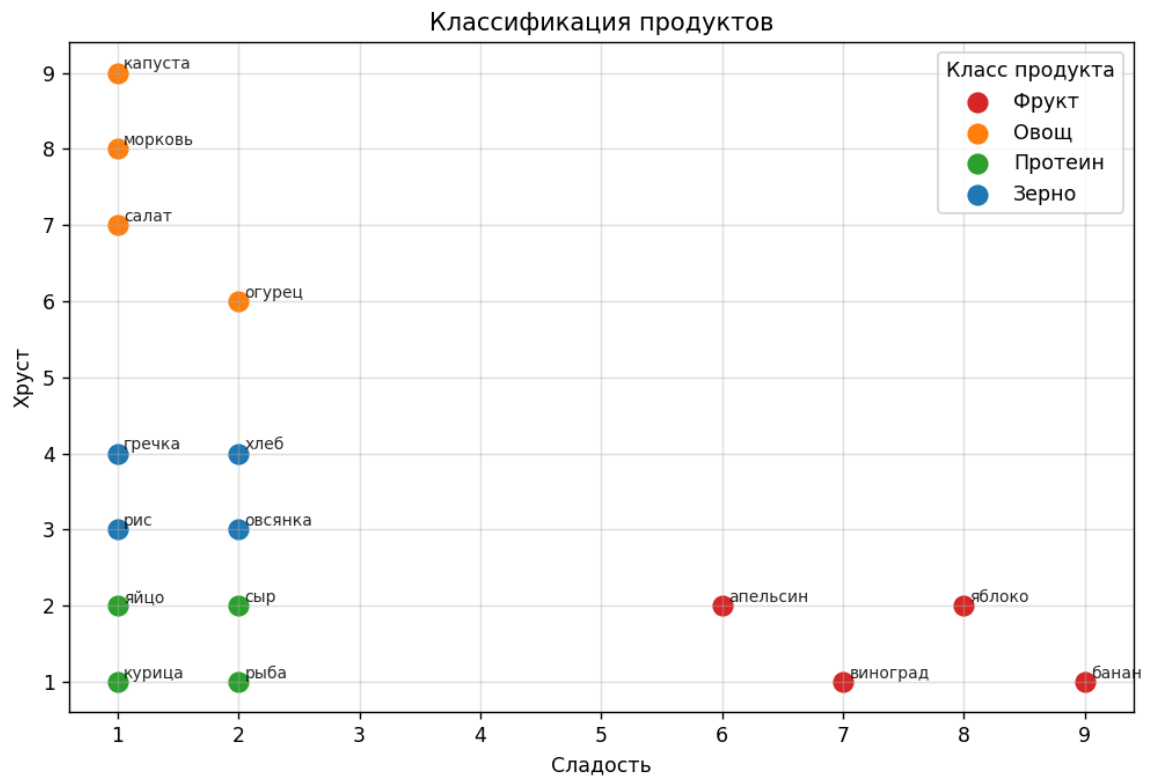
2.Dataset3

Формат строк тот же: продукт сладость хруст Зерно

яблоко 8 2 Фрукт
банан 9 1 Фрукт
виноград 7 1 Фрукт
апельсин 6 2 Фрукт
морковь 1 8 Овощ
салат 1 7 Овощ
огурец 2 6 Овощ
капуста 1 9 Овощ
курица 1 1 Протеин
рыба 2 1 Протеин
яйцо 1 2 Протеин
сыр 2 2 Протеин
хлеб 2 4 Зерно
рис 1 3 Зерно
гречка 1 4 Зерно
овсянка 2 3 Зерно

Параметр эксперимента: $k=5$.

Визуализация: строится график “Классификация продуктов”, где по оси X — сладость, по оси Y — хруст. Точки имеют разные цвета по классам.



- Результаты для dataset3:

Время ручного k-NN: 0.0012057999847456813 сек

Время sklearn k-NN: 0.11213119991589338 сек

Точность (ручной): 0.9375

Точность (sklearn): 0.1

- Вывод по dataset3:

при добавлении нового класса задача усложняется, поэтому точность может измениться;

распределение точек на графике позволяет визуально оценить, насколько классы разделимы.

Исследование k

k	accuracy (ручной)	accuracy (sklearn)
3	0.7333333333333333	0.8666666666666667
5	0.5333333333333333	0.8
7	0.6	0.6

Вывод:

По результатам эксперимента на данном наборе данных наилучшая точность достигается при малых k (для ручного: $k=1$ или $k=3$, для `sklearn`: $k=1$).

Итоговый вывод

В ходе лабораторной работы была реализована ручная версия метрического классификатора k -NN и выполнено сравнение с реализацией из `scikit-learn`. Проведены эксперименты на исходном наборе данных и на расширенном наборе с добавлением нового класса. Визуализация показала распределение объектов на плоскости признаков “сладость–хруст”. В целом оба подхода дают сопоставимые результаты, при этом библиотечная реализация удобнее для масштабирования и дальнейших экспериментов, а ручная — наглядна для понимания принципа работы метода.